

DRAINSinkhorn: Dynamic Compaction for Batched Entropic Optimal Transport*

Xinyang Wen

Abstract

Fast Sinkhorn kernels reduce the cost of each optimal transport update, but a fixed-width batch continues to execute completed problems until its slowest member finishes. We present DRAINSinkhorn, an execution layer that combines a one-marginal screen, a two-marginal verifier, and compaction of unfinished state. A hardware-aware performance model combines observed completion trajectories with independently calibrated kernel and control costs. On MetroPT-3 target groups held out of calibration, its solver-time estimates are within 1.08% of measured time. The primary MetroPT-3 application achieves $1.75\times$ pipeline speedup with four GPU workers. On Packer19, compaction gives $1.54\times$ solver speedup and $1.22\times$ pipeline speedup over full-width logical masking. On ImageNet-32, it achieves $1.05\times$ Sinkhorn speedup and $1.04\times$ over full training. Separate MetroPT-3 controls show that indexed execution and in-place skipping recover similar runtime benefits with less than 0.1% difference from physical compaction. The results locate the benefit in avoiding completed-problem computation, with state layout and checking frequency determining how it is realized.

1 Introduction

Entropic optimal transport (EOT) supplies couplings for industrial sensor analysis, single-cell trajectory inference, and flow-matching training. GPU implementations accelerate Sinkhorn updates through matrix-free operators, tiled reductions, and fused kernels [4, 5, 19]. These applications often require multiple independent couplings, which can share a batched kernel.

The problems in a batch reach the stopping tolerance at different iterations. A fixed-width kernel nevertheless retains every problem until the slowest one finishes. Completed problems continue to occupy kernel lanes and verification bandwidth. Freezing their state after a full-width update preserves this computation. Avoiding it requires an execution change: skipping inactive problems inside the kernel, launching through an active-index list, or packing the unfinished state into a smaller batch.

Completion-aware batching in numerical solvers and inference systems provides the starting point for our design [7, 18, 21]. We adapt it to EOT by using verified comple-

tion to shrink the next update. Sinkhorn problems are independent along the batch dimension, so removing one problem preserves the update applied to the remaining problems. Real workloads contain enough variation in completion iterations for this removal to matter: in the Packer19 component test, physical compaction reduces executed problem updates from 256 to 156 (table 5).

We introduce DRAINSinkhorn, an execution layer that performs screen, verify, and compact during batched Sinkhorn. After a complete scaling cycle, one marginal is current in exact arithmetic. A batched screen evaluates the other marginal and selects candidates for the full two-marginal verifier. The executor emits verified outputs and compacts potentials, marginals, support descriptors, and scratch buffers together. Subsequent kernels process the unfinished batch.

The runtime benefit depends on the GPU cost of the resulting widths. A smaller batch can launch fewer blocks while still occupying the same number of scheduling waves. We therefore model runtime using observed completion iterations, independently measured update time at each active width, and control overhead. On MetroPT-3, calibration from one target group reconstructs solver time on three other groups with errors between +0.463% and +1.08% (section 5.7).

We evaluate this design on MetroPT-3, ImageNet-32, Packer19, and A2D2. The primary MetroPT-3 experiment achieves $1.75\times$ pipeline speedup. ImageNet-32 achieves $1.05\times$ Sinkhorn speedup and $1.04\times$ over full training. Component tests measure screening and removal relative to full-width execution. A separate MetroPT-3 comparison finds less than 0.1% timing difference between physical compaction, indexed execution, and in-place skipping. Width, grouping, and backend experiments identify the conditions under which avoiding completed work reduces runtime.

This paper makes three contributions. First, DrainSinkhorn adapts completion-aware execution to matrix-free EOT through one-marginal screening, two-marginal verification, and aligned state compaction. Second, a calibrated hardware-aware performance model reconstructs runtime from completion trajectories and explains width-dependent costs. Third, application experiments and competitive execution controls distinguish the benefit of skipping completed work from the choice to move its remaining state, alongside task-output and actual-output precision checks.

*Code: github.com/cis-acniticacid/DrainSinkhorn

2 Background and Motivation

2.1 Batched entropic optimal transport

For positive unit-mass marginals $a \in \mathbb{R}_+^n$ and $b \in \mathbb{R}_+^m$, EOT solves

$$\min_{\pi \geq 0} \langle C, \pi \rangle + \varepsilon \sum_{ij} \pi_{ij} (\log \pi_{ij} - 1), \quad \pi \mathbf{1} = a, \quad \pi^\top \mathbf{1} = b. \quad (1)$$

Here C is the transport cost, $\varepsilon > 0$ is the regularization parameter, and π is the coupling. With $K = \exp(-C/\varepsilon)$, Sinkhorn alternates diagonal scalings to form $\pi = \text{diag}(u) K \text{diag}(v)$ [3]. A batched solver applies these updates to several independent problems.

Coordinate, relaxed, low-rank, and Newton methods change the work inside an individual solve [1, 9, 15, 16]. Matrix-free and fused implementations reduce memory traffic, including the stabilized LogSumExp updates of FlashSinkhorn [19]. DrainSinkhorn operates around the selected update kernel and changes the set of problems receiving its next invocation.

2.2 Completed problems consume batch work

Fixed-width execution presents the original batch to every update. When some problems have passed verification, later kernels still contain their lanes. The Packer19 component experiment makes this cost visible: our logical-mask control retains 256 problem-update slots, whereas dynamic compaction executes 156 (table 5). This comparison uses the same observed completion decisions; section 5.4 gives the measurement settings.

Fewer update slots need not produce a proportional reduction in GPU time. The kernel’s grid depends on batch width and problem size, and GPU scheduling groups blocks into waves. A width reduction saves kernel time when it shortens this execution. This motivates both an executor that removes completed problems and a model that accounts for the measured cost at each remaining width.

2.3 Related execution systems

Completion-aware execution has a history in numerical solvers and accelerator systems. At MLSys 2020, Radul et al. [14] batch control-intensive programs by tracking each member’s program position and advancing compatible members together. Orca, presented at OSDI 2022, schedules autoregressive requests at iteration boundaries, returns completed requests, and admits new ones [21]. These systems establish iteration-level scheduling as a general technique for heterogeneous workloads.

Numerical solvers also exploit independent completion. Ginkgo’s batched CG assigns independent systems to workgroups inside a fused kernel, where each system stops at its own residual threshold [7]. Ginkgo’s batched

sparse solvers have also been integrated into plasma simulations [8]. The jaxipm preprint combines heterogeneous interior-point iterations with replacement of completed optimization problems [18]. Within OT, POT provides `solve_batch`, and OTT-JAX supports vectorized Sinkhorn solves [4, 6, 13].

DrainSinkhorn adapts this execution principle to matrix-free EOT. Alternating scaling satisfies the most recently updated marginal in exact arithmetic (equation (3)); the other marginal supplies a one-sided screen for a full two-marginal verifier. Verified completion then triggers aligned selection of support descriptors, marginals, potentials, and scratch state, so the next kernel receives fewer problems. Ginkgo stops per-system work inside a fused solve, and jaxipm replaces completed problems to sustain a batch; DrainSinkhorn drains a finite EOT batch through successively narrower kernels. Its component tests separately measure the OT-specific screen and the physical width reduction.

To study these execution choices in EOT, we implement controls within our Sinkhorn backend. Logical masking is our full-width state-freezing control; indexed execution and in-place skipping test avoidance of completed-problem computation. The cited systems supply related execution ideas, while the measured OT controls are implementations constructed for this study.

Initialization also affects batch execution by changing completion iterations. Carried potentials, analytic extensions, and learned proposals reduce the initial residual [2, 17]; soft c -transforms support changing couplings in flow matching [22]. Our experiments include cold-start couplings and a controlled setting with shared initialization. Downstream tasks consume the resulting coupling, for example to select flow-matching training pairs [12]. Their total runtime includes both the solver and this consumer work.

3 DrainSinkhorn

3.1 Problem and execution state

Consider a batch of W independent EOT problems from equation (1). A lane contains one problem’s marginals, support descriptors, dual potentials, and temporary state. An active problem is one that has yet to pass the configured stopping rule. DrainSinkhorn maintains a packed array of active problems and their output destinations.

The executor advances this array until every problem has completed or the configured iteration limit is reached. Its central invariant is that all tensors at an active lane refer to the same problem. Each emitted output must also pass the two-marginal verifier. These two conditions govern screening, output release, and compaction.

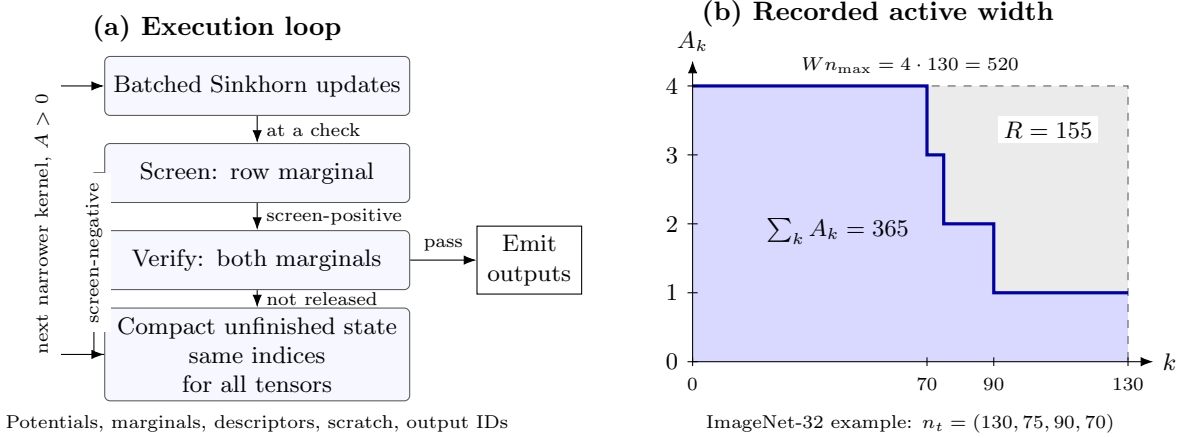


Figure 1: Screen, verify, and compact. Left: screening selects verifier candidates at each configured check; completed outputs are emitted and the remaining tensors share one selection before the next update. Right: one recorded ImageNet-32 window from the width study (Appendix A.5) illustrates the active-width trajectory. Blue area counts executed problem updates; the remaining rectangle area R counts removed updates. This area describes work, while kernel and control costs determine time.

3.2 Screen, verify, and compact

Each iteration applies a batched Sinkhorn cycle to the active problems. At the verification interval, the executor screens every active lane, verifies screen-positive candidates, emits passing outputs, and compacts the unfinished state. The verification interval counts the Sinkhorn cycles between these checks.

For one problem, a Sinkhorn cycle computes

$$u^{k+1} = a \oslash (Kv^k), \quad v^{k+1} = b \oslash (K^\top u^{k+1}), \quad (2)$$

where $\oslash$ denotes elementwise division. Immediately after the second scaling, exact arithmetic gives

$$v^{k+1} \odot (K^\top u^{k+1}) = b. \quad (3)$$

This alternating-scaling identity supplies the OT-specific screen: the updated column marginal satisfies its target, while the row marginal $u^{k+1} \odot (Kv^{k+1})$ may still have a residual. The screen batches this row-residual calculation and selects a problem when $\hat{r}_{t,k}^{\text{row}} \leq \tau_{\text{screen}}$, where τ_{screen} is the screening threshold.

The full verifier recomputes both marginals of each selected coupling $\pi_{t,k}$. The stopping criterion is

$$r_{t,k} = \max \{ \|\pi_{t,k} \mathbf{1} - a_t\|_1, \|\pi_{t,k}^\top \mathbf{1} - b_t\|_1 \} \leq \tau_{\text{stop}}, \quad (4)$$

where τ_{stop} is the stopping tolerance. The timed implementation applies this test to backend-computed residuals. Passing problems are emitted; rejected candidates remain active. A screen false positive adds a verifier call, and a false negative postpones compaction. Independent higher-precision evaluation of the emitted potentials checks the numerical accuracy of this decision in section 5.6.

3.3 Compacting independent state

After verification, the executor applies the same selection to support descriptors, marginals, potentials, centered

shifts, log weights, and scratch buffers. It preserves each problem’s output destination as lanes move. This aligned selection produces the narrower inputs for the next backend call.

Let F_t be one Sinkhorn cycle on the complete state z_t of problem t . Because batching adds a problem dimension with independent updates, ideal arithmetic gives

$$\tilde{F}(\text{pack}(z_1, \dots, z_W)) = \text{pack}(F_1(z_1), \dots, F_W(z_W)). \quad (5)$$

Selecting a subset of the packed problems therefore preserves the update of each remaining problem in exact arithmetic. In floating-point execution, the output verifier applies the backend residual test; section 5.6 separates these checks from independent FP64 replay.

Two alternatives preserve the original state layout. Indexed execution compacts only the active problem IDs and launches a smaller grid whose blocks address the original tensors through those IDs. In-place skipping keeps the original grid and tests activity before performing the dot products and LogSumExp reductions. Both avoid the expensive updates of completed problems. By contrast, the full-width logical-mask control computes updates before restoring completed states. Section 5.7 compares the three compute-skipping implementations with common in-place update buffers.

3.4 Backend execution

The packed Triton implementation applies screening and verification at the configured interval and compacts all matrix-free solver state. A tail threshold controls handoff to its single-problem solver; the primary MetroPT-3 and Packer19 applications set this threshold to one. The competitive execution study keeps the packed kernel through width one for every compared implementation. Iteration limits and non-finite states terminate with failure.

Runtime includes the configured control and handoff operations.

The OTT-JAX implementation verifies at fixed phase boundaries, removes completed problems, and rebuckles the remaining batch. PyKeOps uses a batched independent-problem operator with a fixed checking interval and removal of completed state. When a problem fits on one GPU, independent workers process different windows. The scale-out experiments use one worker per GPU.

4 Hardware-Aware Performance Model

The model connects completion iterations to executed work and runtime. Its inputs are the observed completion iteration of each problem, independently measured update costs, and calibrated control overhead. This use of workload shape and device cost follows accelerator performance modeling [10, 20]. Work-count identities supply the workload inputs; device measurements supply the costs. Section 5.7 tests the resulting time estimates against separate application runs.

4.1 Completion iterations determine active width

Let n_t be the iteration of the first configured verification at which problem t passes, and let $n_{\max} = \max_t n_t$. Before update k , the active width is

$$A_k = |\{t : n_t \geq k\}|. \quad (6)$$

These iteration counts include the effect of the verification interval. A more frequent schedule can observe completion earlier and produce a different trajectory.

For the same completion decisions, fixed-width execution uses $Wn_{\max}$ problem-update slots. Compaction uses

$$S_{\text{active}} = \sum_{k=1}^{n_{\max}} A_k = \sum_{t=1}^W n_t, \quad S_{\text{static}} = Wn_{\max}. \quad (7)$$

The identity follows by counting each problem once at every iteration before its completion. The removed work is

$$R = Wn_{\max} - \sum_t n_t. \quad (8)$$

As illustrated in figure 1, R is the area between the fixed-width rectangle and the active-width trajectory. In the full-width logical-mask control, completed states are frozen while their update computations remain. Indexed execution and in-place skipping can also attain $\sum_t n_t$ expensive problem updates while preserving the original state layout.

A batch has removable work when completed and unfinished problems coexist before an update. Thus variation

within a batch matters: identical completion iterations yield $R = 0$, even if completion iterations differ across batches. The grouping experiment changes this within-batch variation using a fixed set of problems.

4.2 Kernel savings must cover control cost

Let $c_k(a, \xi_k)$ denote the measured cost of update k at width a . The state ξ_k specifies support shape, data type, kernel configuration, and other conditions of the comparison. The compacted runtime is

$$T_{\text{active}} = \sum_k c_k(A_k, \xi_k) + H_{\text{screen}} + H_{\text{verification}} + H_{\text{compact}} + H_{\text{other}}. \quad (9)$$

The H terms account for screening, verification, compaction, and remaining timed control work. Write their sum as H_{active} , and let $\Delta H = H_{\text{active}} - H_{\text{fixed}}$ be the incremental control cost relative to fixed-width execution.

For a comparison with the same completion decisions and matched update states, subtracting the two runtime accounts gives

$$T_{\text{fixed}} - T_{\text{active}} = \sum_{k=1}^{n_{\max}} [c_k(W, \xi_k) - c_k(A_k, \xi_k)] - \Delta H. \quad (10)$$

Compaction reduces runtime when

$$\sum_k [c_k(W, \xi_k) - c_k(A_k, \xi_k)] > \Delta H. \quad (11)$$

The width sweep measures the update-cost term, and component measurements quantify the additional control work. This equation explains the break-even condition for the measured execution path.

4.3 Calibration and prediction

A runtime estimate for a new batch uses independently calibrated costs $\hat{c}(a, \xi)$ and overheads $\hat{H}$. Given a completion trace, its conditional speedup estimate is

$$\hat{S} = \frac{n_{\max} \hat{c}(W, \xi) + \hat{H}_{\text{fixed}}}{\sum_k \hat{c}(A_k, \xi) + \hat{H}_{\text{active}}}. \quad (12)$$

Calibration fixes the device, precision, support shape, tile configuration, state layout, and tail path. Costs are specific to the execution strategy: indexed access, entry skipping, and physical compaction can have different update and control costs even for the same active width.

For solver-time validation, we measure update and screen costs at every active width and profile initialization, verification, and release processing in separate calibration

runs. The estimate is

$$\begin{aligned} \hat{T} = & \hat{H}_{\text{init}} + \sum_k \hat{c}_{\text{update}}(A_k, \xi) \\ & + \sum_{j \in \mathcal{J}} \left[\hat{c}_{\text{screen}}(A_j, \xi) + \hat{h}_{\text{verify}}(v_j) + \hat{h}_{\text{release}}(d_j) \right], \end{aligned} \quad (13)$$

where $\mathcal{J}$ indexes checks, v_j counts verifier candidates, and d_j counts released problems. Verification uses a calibrated per-candidate cost; release processing uses a calibrated event cost, with separate costs for empty checks. The observed trajectory and event counts determine the estimate, while the target run’s total time is reserved for evaluation. We report signed error $100(\hat{T}/T - 1)$ in section 5.7. Target-group transfer holds the evaluated targets out of calibration and retains the shared source input. Pipeline estimates additionally require consumer costs under the same configuration.

4.4 GPU scheduling makes width cost stepwise

For the packed Triton update, width a and support size n produce

$$B(a, n) = a \left\lceil \frac{n}{b_m} \right\rceil \quad (14)$$

thread blocks, where b_m is the row tile size. GPU scheduling executes blocks in waves whose capacity depends on occupancy [10, 11]. A model of this cost is

$$c(a; n, \xi) \approx g_{n, \xi} \left(\left\lceil \frac{B(a, n)}{Q_\xi} \right\rceil \right), \quad (15)$$

where Q_ξ is the resident block capacity for the device and kernel, and $g_{n, \xi}$ maps wave count to elapsed time. Registers and shared memory affect Q_ξ .

Reducing width within a scheduling wave can leave update time nearly unchanged. Larger problems launch more blocks per lane, so removing a lane can reduce the number of waves earlier. Section 5.3 measures this relation on A100 and uses it to interpret the application results.

5 Experiments

The evaluation asks how much DrainSinkhorn reduces application runtime, how the implementation of completed-problem removal affects that gain, and how accurately calibrated costs reconstruct runtime. We report application results, width measurements, component and grouping tests, numerical checks, and competitive execution controls.

5.1 Experimental setup

We evaluate four tasks: MetroPT-3 for industrial sensor streams, ImageNet-32 for flow-matching training,

Packer19 for single-cell trajectory inference, and A2D2 for LiDAR point-cloud matching. Table 1 gives the primary configurations. The packed Triton studies use NVIDIA A100-SXM4-80GB GPUs and verify every four Sinkhorn cycles. ImageNet-32 training uses PCA500 feature couplings, three paired seeds, and 50,000 training updates; generation is evaluated at 4, 8, and 16 function evaluations (NFE).

We measure E_1 (Sinkhorn time) through the verified coupling or dual potentials, E_2 (pipeline time) through the downstream consumer, and E_3 (total time) through the full training, generation, or task-evaluation interval. Speedup is baseline time divided by DrainSinkhorn time. In the two-host MetroPT-3 and Packer19 component tests, E_1 is obtained by subtracting paired consumer time within each observation before computing ratios and uncertainty.

Fixed-width execution keeps the original batch through the final verification. We implement logical masking (LM) as an experimental control in the same OT backend: it computes full-width updates and freezes completed states. Dynamic compaction (DC) removes completed state and narrows the batch. Fixed/DC measures the complete execution change; LM/DC measures removal relative to full-width computation under matched release decisions. Our indexed-execution and in-place-skipping implementations provide the compute-skipping controls in section 5.7. Component tests fix inputs, initialization, tolerance, verification schedule, verifier, backend, and consumer. Timing repeats and host/order/GPU placements characterize measurement variability; the number and identity of input workloads are reported separately.

The competitive MetroPT-3 study uses FP32 with TF32 disabled, $\tau_{\text{screen}} = \tau_{\text{stop}} = 9.5 \times 10^{-4}$, and a backend two-marginal verifier every four cycles. An external tolerance of 10^{-3} is used for post-run FP64 replay. The timed path uses the backend verifier without the outward-bound guard or FP64 escalation. Appendix A.7 gives its kernel configuration, shared inputs, and calibration procedure.

5.2 Application runtime and backend transfer

DrainSinkhorn reduces both solver time and application time, with the size of the benefit depending on the workload. Table 2 separates the three timing levels. The primary MetroPT-3 pipeline achieves $1.75\times$ speedup with four independent GPU workers as problem updates fall from 5696 to 3128. The corresponding ratio of fixed to compacted GPU energy is $1.78\times$. This experiment measures one fixed failure-window workload under three execution orders. The grouping intervention in section 5.5 examines batch-assignment sensitivity on a fixed pool of 32 other MetroPT-3 problems.

Packer19 shows how solver savings translate to the pipeline. In the same 24 paired observations, LM/DC is $1.54\times$ for Sinkhorn time and $1.22\times$ through the transition consumer. The shared consumer work reduces the pipeline

Table 1: Primary task settings. The packed Triton paths use cold initialization. The Packer19 tolerance sweep is reported in Section 5.4; observation units are specified with each study.

Task	Backend	Endpoint	n	W	Numerical settings
MetroPT-3	Triton	Pipeline	16384	16	$\varepsilon = 0.1, \tau = 10^{-3}$
ImageNet-32	Triton	Training	1024	8	$\varepsilon = 0.03, \tau = 10^{-3}$
Packer19	Triton	Transition	16384	8	$\varepsilon = 0.1, \tau = 10^{-3}$
A2D2	PyKeOps	Matching	8192	8	Three LiDAR clips

Table 2: Application results grouped by timing level. Ratios are baseline/DC. Ranges in parentheses describe observed seeds or execution orders; conditional timing intervals are reported in Appendix A.1. Each row retains its own observations and is not pooled with other rows.

Scope	Task	Baseline	Speedup	Observations
E_1	ImageNet-32	Fixed	1.05 (1.03–1.07)	3 training seeds
E_1	Packer19 components	LM	1.54	24 paired units
E_2	MetroPT-3	Fixed	1.75	1 workload; 3 orders
E_2	Packer19 components	LM	1.22	Same 24 paired units
E_3	ImageNet-32	Fixed	1.04 (1.03–1.04)	3 training seeds

ratio. ImageNet-32 gains are smaller: across three paired training seeds, the Sinkhorn-time ratio is $1.05\times$ and the full-training ratio is $1.04\times$. Table 4 reports feature-space quality after training, and table 8 gives absolute solver times by seed.

Backend results separate the contribution of compaction from the benefit of a complete execution configuration (table 3). In OTT-JAX, compaction gives $1.37\text{--}1.58\times$ over fixed-phase execution at tolerances from 10^{-3} to 5×10^{-3} . At 10^{-2} , every problem completes in the first phase and the ratio is $0.995\times$. The native-reference ratio remains $1.73\times$ there because it also includes the change to fixed-phase execution.

On PyKeOps, the earlier configuration comparison favors static batching over compaction with less frequent checking. Checking every four updates reaches 16 rounds, compared with 13 for the static batch, and achieves $1.42\times$ over sequential execution versus $1.55\times$ for static batching. A separate comparison now fixes every method’s checking interval to one (table 3). Across three inputs per size, static/compacted coupling-preparation time is $0.999\times$ at $n = 2048$ and $1.02\times$ at $n = 16384$. At the larger size, all methods take 13 batch rounds; compaction executes 50, 51, and 52 problem updates against 52 for static batching. The third input completes simultaneously and has a ratio of $1.00\times$ after rounding. Thus the matched comparison shows a small input-dependent benefit. Supplementary settings and deployment results appear in appendices A.3 and A.4.

5.3 Kernel time by active batch width

The width experiment tests the update-cost term in equation (10). It varies active width while holding the packed Triton update and problem size fixed. Each point in fig-

ure 2 summarizes 30 direct primitive calls; the timing covers the two update kernels. Screening, verification, and compaction are measured in the component study.

At $n = 1024$, widths one through six have similar update time, followed by a large increase from six to seven. At $n = 4096$, the first large increase occurs from one to two; the $n = 16384$ curve responds from width one. With row tile size $b_m = 64$ and 108 SMs, the one-block-per-SM wave-width estimate $b_m N_{\text{SM}}/n$ is 6.75, 1.69, and 0.42 for these three sizes. The observed steps are consistent with the scheduling-wave explanation.

The small-problem plateau helps interpret the modest ImageNet-32 solver gain: lane removal within the plateau reduces update slots with little change in primitive time. Larger problems have a stronger width response, consistent with the larger MetroPT-3 gains. The application endpoint additionally includes control and consumer work, as expressed in equation (10). The separate initial-width study in figure 4 shows how padding and complete-configuration runtime vary over $W = 1, 2, 4, 8, 16$.

5.4 Packer19 single-variable component tests

The Packer19 study asks which parts of the execution layer reduce time. It uses one transition workload on two hosts with four A100 GPUs each. Three method orders produce 24 paired observations indexed by host, order, and GPU. Timings are medians after warm-up. As shown in Table 5, each contrast changes one component at tolerance 10^{-3} .

Batched verification and the one-marginal screen reduce OT-stage time by factors of 1.13 and 1.11, respectively. Screening lowers the cost of finding completed problems; removal of their later updates produces the larger timing

Table 3: Backend comparisons on ImageNet-32. Top: OTT-JAX ($n = 4096, W = 16$), ten repeats per tolerance; fixed-phase and compacted execution share phase length q . Middle: earlier PyKeOps configurations ($n = 16384, W = 4$), with means and ranges of three sequential/configuration ratios. Bottom: separate PyKeOps runs with every-update checking and $W = 4$; entries are means of three per-input median paired coupling-preparation ratios over three cyclic orders. The lower panels use their own measured baselines.

OTT-JAX: matched-phase compaction and native-reference comparisons			
Tolerance	Phase length q	Fixed-phase / DC	Native / DC
10^{-3}	20	1.58	2.64
3×10^{-3}	10	1.56	2.62
5×10^{-3}	10	1.37	2.30
10^{-2}	10	0.995	1.73
PyKeOps: complete configurations relative to sequential execution			
Configuration	Mean speedup	Seed range	Max. marginal L_1
Static batch	1.55	1.45–1.69	8.80×10^{-4}
Logical mask	1.55	1.45–1.69	8.93×10^{-4}
Compaction, check every 2 updates	1.51	1.43–1.59	8.93×10^{-4}
Compaction, check every 4 updates	1.42	1.36–1.45	8.93×10^{-4}
PyKeOps: every-update checking for all three methods			
Support size n	Static / DC	Mask / DC	Inputs $\times$ orders
2048	0.999	1.00	3×3
16384	1.02	1.02	3×3

Table 4: ImageNet-32 quality after 50,000 updates in PCA500 feature space. Entries are mean $\pm$ sample standard deviation over three paired training seeds; lower values are better. Held-out FM MSE is independent of sampling NFE.

NFE	Execution	Curvature	Projected Fréchet
4	Fixed	5.45 ± 0.0645	1.30 ± 0.348
	Compacted	5.44 ± 0.0504	1.29 ± 0.336
8	Fixed	2.84 ± 0.0238	1.53 ± 0.306
	Compacted	2.84 ± 0.0211	1.52 ± 0.295
16	Fixed	1.45 ± 0.0130	1.82 ± 0.264
	Compacted	1.45 ± 0.0121	1.81 ± 0.257
FM MSE, fixed		1.08 ± 0.0159	
FM MSE, compacted		1.08 ± 0.0157	

Table 5: Single-variable Packer19 component comparisons at $\tau = 10^{-3}$. Ratios use OT-stage time after paired consumer subtraction. All four comparisons use 24 paired observations. The first two contrasts report paired wins; bootstrap intervals for the final two are detailed in Appendix A.1.

Change	Ratio	Evidence
Batched verification	1.13	24/24 faster
One-marginal screen	1.11	24/24 faster
Logical mask	1.00	CI includes 1
Physical compaction	1.54	CI above 1

effect relative to full-width computation. All 24 paired observations favor each change. The screen’s replay checks are reported in section 5.6.

Logical masking leaves the packed width at eight and yields a ratio near one. Compaction reduces problem updates from 256 to 156 and gives $1.54\times$ speedup over this full-width control, with timing uncertainty summarized in appendix A.1. The comparison with compute-skipping implementations is measured separately on MetroPT-3 in section 5.7. Across paired-observation medians, GPU energy falls from 880 to 688 J, while peak allocated memory increases from 80.8 to 107 MiB. Table 9 gives the separately summarized phase times.

A separate stopping-tolerance sweep tests the same LM/DC comparison over five tolerances. Figure 3 shows speedup alongside achieved residual and consumer total-variation distance (TV) from the tight reference. Compaction gives $1.48\text{--}1.64\times$ from 10^{-4} through 3×10^{-3} . At 10^{-2} , all eight problems first pass at update 8; the ratio is $0.999\times$ with a timing interval spanning equal runtime (appendix A.1). This equal-completion case exercises the $R = 0$ condition in the model.

5.5 MetroPT-3 grouping test

The grouping test asks whether completion variation within a batch changes compaction benefit. It holds 32 real MetroPT-3 problems fixed and changes their assignment to windows. Offline every-cycle completion iterations q_t define the grouping; verifier-derived iterations n_t measure the executed work. Homogeneous grouping places prob-

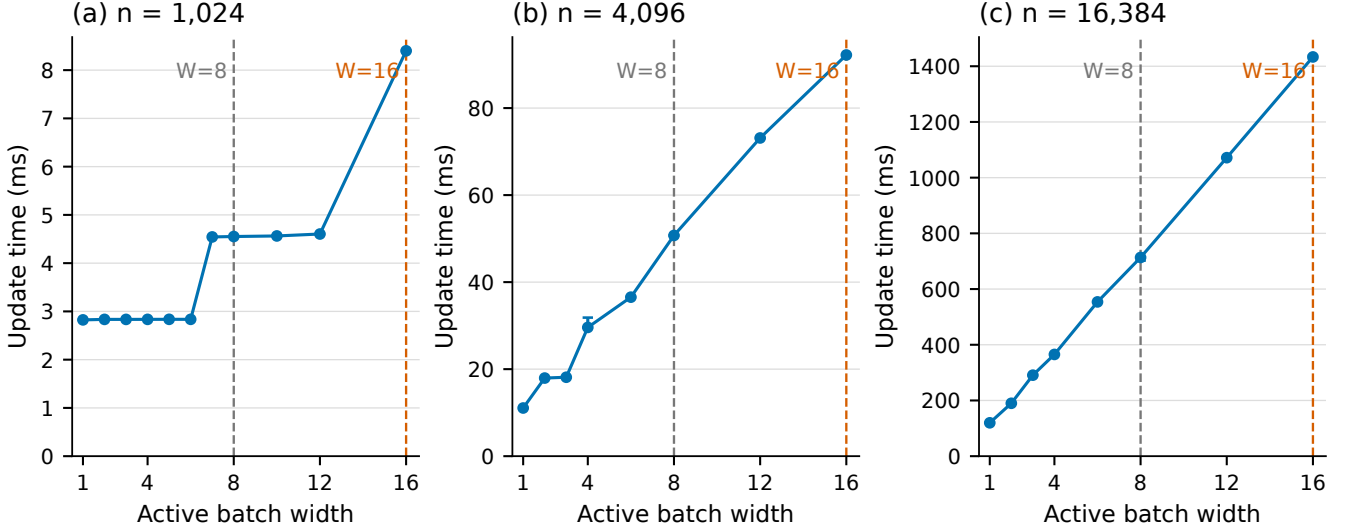


Figure 2: A100 packed-update time versus active batch width on synthetic tensors with feature dimension 500, column tile 64, and cost scale 1. Points show medians and interquartile ranges from 30 repeats, with panel-specific millisecond scales. Vertical guides locate $W = 8$ and $W = 16$. The sweep characterizes the width response of this configuration; the application-time reconstruction in section 5.7 uses its own matched calibration.

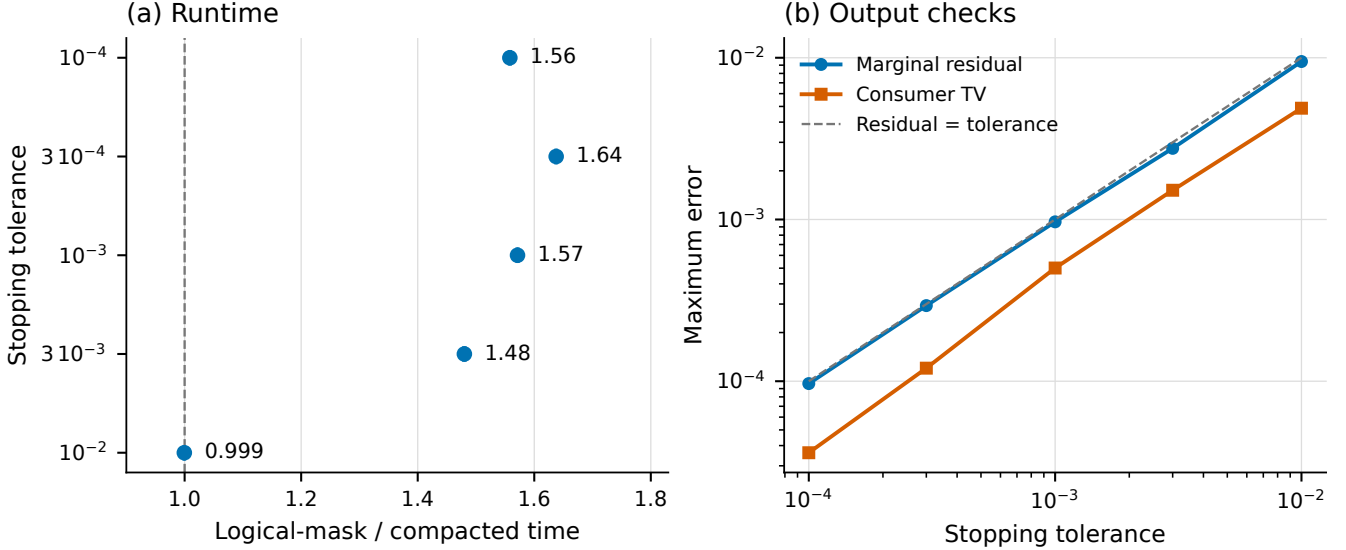


Figure 3: Packer19 compaction across stopping tolerances. Left: LM/DC speedup with equal-host stratified bootstrap 95% intervals over 24 paired observations per tolerance; the vertical line marks equal runtime. Right: maximum achieved marginal residual and maximum consumer TV against the tight reference. The dashed diagonal marks residual equal to the requested tolerance.

lems with similar completion iterations together, while heterogeneous and random layouts create more completed lanes inside unfinished batches.

As shown in Table 6, the fixed/DC ratio is $1.10\times$ for homogeneous grouping, $1.22\times$ for heterogeneous grouping, and $1.23\text{--}1.27\times$ for random layouts. The corresponding dynamic-update fractions are 0.892, 0.802, and 0.764–0.789. These comparisons show how batch assignment changes removable work for the same problem set.

5.6 Numerical accuracy and application outputs

The timed packed Triton applications use backend-computed two-marginal residuals for release. On primary MetroPT-3, maximum recorded row and column residuals are 9.5×10^{-4} and 1.3×10^{-6} ; both execution modes attain failure-window ROC AUC 1.0. ImageNet-32 quality measurements are close across the three paired seeds

Table 6: Effect of grouping 32 fixed MetroPT-3 problems. The update fraction is dynamic/fixed problem updates; speedup is fixed/dynamic time. Both quantities summarize execution of the same problem set under each assignment.

Grouping	Update fraction	Speedup
Homogeneous	0.892	1.10
Heterogeneous	0.802	1.22
Random 1	0.776	1.26
Random 2	0.789	1.23
Random 3	0.764	1.27

in PCA500 feature space at NFE 4, 8, and 16 (table 4). These measurements describe the reported feature-space metrics.

Screening and release are checked separately. The full-verifier replay of 1488 screen-negative events finds zero completed problems missed in this component workload; this measures screening delay at its configured tolerance. In the matched Packer19 LM/DC comparison, release decisions agree and the consumer TV difference is zero. The tolerance sweep in figure 3 reports achieved residuals and consumer deviations from a tighter reference.

The competitive MetroPT-3 study additionally saves the potentials emitted by the first timed run of every method and batch. Blocked FP64 direct-dot reconstruction evaluates both marginals of these actual outputs after timing. All 96 method-problem checks satisfy the external 10^{-3} tolerance, with maximum residual 9.5×10^{-4} . They cover 16 distinct target problems sharing one source; the larger batch and method copies reuse those inputs. Saved potentials agree bitwise across methods within each batch.

Finite-precision checks also exercise cases near the acceptance threshold. An independent stress test covers 84 near-threshold problems and 16 runtime cases. Raw FP32 and TF32 each disagree with FP64 replay on three acceptance decisions; an outward-bound guard with FP64 escalation has zero disagreements. The available summary reports total disagreements without separating false acceptance from false rejection. Appendix A.6 specifies this separately evaluated guarded configuration.

5.7 Execution alternatives and calibrated runtime

We compare physical compaction with indexed execution and in-place skipping on one A100 using the existing MetroPT-3 failure-window input. Its 16 targets form four non-overlapping groups of four; a width-eight batch reuses the first eight targets. Each batch uses four cyclic method orders. The three primary implementations share in-place Gauss-Seidel buffers, screening, verification, and a packed width-one tail. Indexed execution changes only the active IDs and launch grid; in-place skipping retains the original

grid and bypasses expensive updates for inactive problems. Both preserve the original state storage. A legacy logical-mask implementation is a secondary reference because it allocates new update outputs before restoring inactive state.

Table 7 reports absolute solver times. Paired index/DC ratios correspond to a 0.0500–0.0711% timing disadvantage for DC; entry-skip/DC favors DC by 0.0168–0.0469%. All differences are below 0.1%; skipping completed computation gives similar measured runtime across these three state-management choices.

The same runs test equation (13). Independent calibration measures every active width with five primitive repeats and obtains control costs from a separate instrumented solve. Calibration on the first width-four group is transferred to the other three groups. Across the three primary methods and their four timing repeats, signed reconstruction errors are +0.463% to +1.08%. These estimates use the observed completion and verification traces and shared-source inputs. Prefix-ID calibration approximates the noncontiguous survivor IDs used during indexed execution.

An earlier scheduling study uses the same source and targets together at width 16. It compares immediate exact compaction with graph replay, periodic compaction, and bucketed capacities. Immediate compaction takes 8.43 s; four-cycle graph replay also takes 8.42 s, while bucketed execution takes 9.24 s. Its independently measured width and movement costs explain why saving a small amount of movement can add more padded update work. That study tests when and how far to compact. The present comparison tests whether full-state movement is needed to avoid completed updates; its regrouped inputs add execution contrasts to the existing workload evidence.

6 Discussion

DrainSinkhorn benefits batches whose members complete at different verification rounds and whose narrower kernels save enough time to cover screening, verification, and state movement. The grouping test and width sweep examine these two conditions separately. Initialization and batch assignment affect completion iterations, while problem size and kernel configuration determine the time associated with each active width.

Physical compaction is one implementation of completed-problem removal. The Packer19 comparison with full-width logical masking measures lower runtime and energy together with 26.1 MiB of additional peak allocation. On the separate MetroPT-3 controls, indexed execution and in-place skipping reach nearly the same runtime while preserving state storage. These results favor selecting state layout and launch strategy by measured backend cost. The direct compute-skipping comparison currently covers the selected MetroPT-3 inputs.

The performance model uses a width-cost function for a specified device, kernel, problem shape, precision, and

Table 7: MetroPT-3 compute-skipping controls and conditional runtime reconstruction on one A100. Times are median continuous solver wall times over four cyclic orders, in seconds. Error ranges span three methods and four repeats and use $100(\bar{T}/T - 1)$. Calibration uses targets 0–3 at $W = 4$ and 0–7 at $W = 8$ in separate runs. Transfer rows hold the target group out of calibration; all groups share one source.

Targets	W	Indexed (s)	Entry skip (s)	DC (s)	Error (%)	Calibration use
0–3	4	4.56	4.56	4.56	+0.574–+0.598	Same input, separate runs
4–7	4	1.00	1.00	1.00	+1.03–+1.08	Target-group transfer
8–11	4	1.32	1.32	1.32	+0.889–+1.01	Target-group transfer
12–15	4	4.68	4.68	4.68	+0.463–+0.566	Target-group transfer
0–7	8	5.33	5.34	5.34	+0.483–+0.56	Same input, separate runs

execution strategy. The MetroPT-3 target-group transfer validates conditional time reconstruction under this fixed configuration. The synthetic sweep separately characterizes scheduling-wave behavior. Applying the model to another backend requires its own width and control calibration, including the tail path. Completion counts support offline analysis of grouping; estimating them for online scheduling remains a separate problem.

7 Conclusion

DrainSinkhorn combines Sinkhorn-specific screening with verification and compaction to avoid completed-problem updates. Application experiments measure the resulting solver, pipeline, and training gains. Competitive MetroPT-3 controls show that indexed execution and in-place skipping achieve similar runtime to full-state compaction. Independently calibrated costs reconstruct solver time on held-out target groups from their observed completion traces. Together, the results identify completion-aware execution as the source of savings and measured backend cost as the guide to its implementation.

References

- [1] Jason Altschuler, Jonathan Niles-Weed, and Philippe Rigollet. Near-linear time approximation algorithms for optimal transport via Sinkhorn iteration. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL https://papers.nips.cc/paper_files/paper/2017/hash/491442df5f88c6aa018e86dac21d3606-Abstract.html.
- [2] Brandon Amos, Giulia Luise, Samuel Cohen, and Ievgen Redko. Meta optimal transport. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 791–813, 2023. URL <https://proceedings.mlr.press/v202/amos23a.html>.
- [3] Marco Cuturi. Sinkhorn distances: Lightspeed computation of optimal transportation distances. In *Advances in Neural Information Processing Systems*, volume 26, pages 2292–2300, 2013. URL <https://arxiv.org/abs/1306.0895>.
- [4] Marco Cuturi, Laetitia Meng-Papaxanthos, Yingtao Tian, Charlotte Bunne, Geoff Davis, and Olivier Teboul. Optimal transport tools (OTT): A JAX toolbox for all things wasserstein, 2022. URL <https://arxiv.org/abs/2201.12324>.
- [5] Jean Feydy, Thibault Séjourné, François-Xavier Vialard, Shun ichi Amari, Alain Trounev, and Gabriel Peyré. Interpolating between optimal transport and MMD using sinkhorn divergences. In *Proceedings of the 22nd International Conference on Artificial Intelligence and Statistics*, volume 89 of *Proceedings of Machine Learning Research*, pages 2681–2690, 2019. URL <https://proceedings.mlr.press/v89/feydy19a.html>.
- [6] Rémi Flamary, Nicolas Courty, Alexandre Gramfort, Mokhtar Z. Alaya, Aurélie Boissunon, Stanislas Chambon, Laetitia Chapel, Adrien Corenflos, Kilian Fatras, Nemo Fournier, Léo Gautheron, Nathalie T. H. Gayraud, Hicham Janati, Alain Rakotomamonjy, Ievgen Redko, Antoine Rolet, Antony Schutz, Vivien Seguy, Danica J. Sutherland, Romain Tavenard, Alexander Tong, and Titouan Vayer. POT: Python optimal transport. *Journal of Machine Learning Research*, 22(78):1–8, 2021. URL <https://www.jmlr.org/papers/v22/20-451.html>.
- [7] Ginkgo Project. Batched CG. Ginkgo user guide, 2026. URL <https://gko-project.org/user-guide/concepts/batched/cg.html>. Accessed 2026-09-08.
- [8] Aditya Kashi, Pratik Nayak, Dhruva Kulkarni, Aaron Scheinberg, Paul Lin, and Hartwig Anzt. Integrating batched sparse iterative solvers for the collision operator in fusion plasma simulations on gpus. *Journal of Parallel and Distributed Computing*, 2023. URL <https://publikationen.bibliothek.kit.edu/1000158257/156817569>.
- [9] Mete Kemertas, Amir massoud Farahmand, and Allan D. Jepson. A truncated newton method for optimal transport, 2025. URL <https://arxiv.org/abs/2504.02067>.
- [10] *GPU Performance Background User’s Guide*. NVIDIA, 2022. URL <https://docs.nvidia.com/deeplearning/performance/dl-performance-gpu-background/index.html>.
- [11] *CUDA C++ Programming Guide*. NVIDIA, 2024. URL <https://docs.nvidia.com/cuda/archive/12.5.0/cuda-c-programming-guide/index.html>.

- [12] Aram-Alexandre Pooladian, Heli Ben-Hamu, Carles Domingo-Enrich, Brandon Amos, Yaron Lipman, and Ricky T. Q. Chen. Multisample flow matching: Straightening flows with minibatch couplings. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 28100–28127, 2023. URL <https://proceedings.mlr.press/v202/pooladian23a.html>.
- [13] POT developers. POT 0.9.6 release notes. Python Optimal Transport documentation, 2025. URL <https://pythonot.github.io/releases.html>.
- [14] Alexey Radul, Brian Patton, Dougal Maclaurin, Matthew D. Hoffman, and Rif A. Saurous. Automatically batching control-intensive programs for modern accelerators. In *Proceedings of Machine Learning and Systems*, volume 2, 2020. URL https://proceedings.mlsys.org/paper_files/paper/2020/hash/39945d578f616735572174bf5e8f155d-Abstract.html.
- [15] Meyer Scetbon, Marco Cuturi, and Gabriel Peyré. Low-rank Sinkhorn factorization. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 9344–9354, 2021. URL <https://proceedings.mlr.press/v139/scetbon21a.html>.
- [16] Alexis Thibault, L  na  c Chizat, Charles Dossal, and Nicolas Papadakis. Overrelaxed Sinkhorn–Knopp algorithm for regularized optimal transport, 2017. URL <https://arxiv.org/abs/1711.01851>.
- [17] James Thornton and Marco Cuturi. Rethinking initialization of the sinkhorn algorithm. In *Proceedings of the 26th International Conference on Artificial Intelligence and Statistics*, volume 206 of *Proceedings of Machine Learning Research*, pages 8682–8698, 2023. URL <https://proceedings.mlr.press/v206/thornton23a.html>.
- [18] John Viljoen, Johanna Haffner, Masayoshi Tomizuka, and Negar Mehr. Scaling nonlinear optimization: Many problems one GPU, 2026. URL <https://arxiv.org/abs/2606.26341>.
- [19] Felix X.-F. Ye, Xingjie Li, An Yu, Ming-Ching Chang, Linsong Chu, and Davis Wertheimer. FlashSinkhorn: IO-aware entropic optimal transport on GPU, 2026. URL <https://arxiv.org/abs/2602.03067>.
- [20] Geoffrey X. Yu, Yubo Gao, Pavel Golikov, and Gennady Pekhimenko. Habitat: A runtime-based computational performance predictor for deep neural network training. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*, pages 503–521, 2021. URL <https://www.usenix.org/conference/atc21/presentation/yu>.
- [21] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation*, pages 521–538, 2022. URL <https://www.usenix.org/conference/osdi22/presentation/yu>.
- [22] Stephen Zhang, Alireza Mousavi-Hosseini, Michal Klein, and Marco Cuturi. On fitting flow models with large sinkhorn couplings, 2025. URL <https://arxiv.org/abs/2506.05526>.

A Measurement Details

A.1 Timing and statistical units

The primary MetroPT-3 endpoint is the maximum pipeline time across four independent GPU workers. Each of three execution orders is summarized by the median of two timing repeats. The reported $1.75\times$ is the descriptive geometric mean of the three order ratios. Both paths use the same input, initialization, regularization, tolerance, verification schedule, backend, and consumer. Each repeat invokes and evaluates 16 verifier candidates and launches 32 plan-application kernels.

The two-host MetroPT-3 comparison uses three order-balanced plans on each host, with four GPU placements per host. Its six host/plan combinations are paired timing units for a fixed workload; placements and repeats stabilize timing. The E_2 ratio is $1.74\times$, with a stratified bootstrap interval of $[74.3, 74.7]\%$ for the relative timing ratio increase, $100(S-1)$. Subtracting each unit’s paired consumer time gives the E_1 ratio $1.75\times$. The primary four-worker and two-host comparisons use their own endpoints and observations.

Ratios are displayed to three significant digits. For narrow timing intervals, we report $100(S-1)$ in percent to retain their resolution: the Packer19 component intervals are $[53.9, 54.0]\%$ for E_1 and $[22.4, 22.5]\%$ for E_2 . The logical-mask comparison has interval $[-0.019, 0.033]\%$; the simultaneous-completion stopping case has interval $[-0.134, 0.0320]\%$. These bootstrap intervals describe timing variation for the fixed inputs and host/order/GPU units. Across-seed training variation is reported separately as sample standard deviation.

GPU energy is the change in the A100 NVML cumulative energy counter between the start of the method call and the final CUDA synchronization. The counter reports millijoules, converted to joules. This interval starts with prepared arrays and includes solver, initialization, and consumer work; compilation, warm-up, and prepared-input transfer occur before it. Unsupported counters are recorded as unavailable.

A.2 Training and output measurements

ImageNet-32 uses paired seeds 20260891, 20260892, and 20260893 throughout training and NFE evaluation. Table 8 reports each seed’s solver time. The full-training ratio uses the complete 50,000-update interval. The separate coupling-preparation diagnostic gives paired ratios 1.16, 1.08, and 1.08, with fixed/DC update counts 64/52, 64/56, and 64/56.

Table 8: ImageNet-32 solver times by training seed. The geometric mean of paired fixed/compacted ratios is 1.05.

Measurement	Seed 20260891	Seed 20260892	Seed 20260893
Fixed time (s)	123	124	119
Compacted time (s)	117	116	115
Fixed / compacted	1.06	1.07	1.03

All 48 worker-level primary MetroPT-3 repeats converge. The solver receives sensor inputs without failure labels. Maximum paired differences in transport cost and barycentric shift are 0.0279 and 0.00347 under the fixed failure-window consumer.

The Packer19 tolerance study uses two hosts, four GPUs per host, three method orders, and three timing repeats. Each tolerance has 24 paired observations indexed by host, order, and GPU, with an equal-host bootstrap interval. Consumer TV compares outputs to the 10^{-6} reference. Absolute OT-stage medians at 10^{-3} are 2.04 s for logical masking and 1.30 s for compaction in this stopping study. The component study’s phase medians in table 9 are summarized independently and belong to its own LM/DC comparison.

Table 9: Packer19 component-study phase times in milliseconds. Entries are separately computed phase medians. Whole-path ratios in Table 5 come from paired timings.

Execution	Correction	Batched verification	Verifier	Mask / compact
Logical mask	1706	639	105	0.306
Compacted	1071	401	105	3.37

A.3 Width measurements and backend settings

The width sweep uses an A100-SXM4-80GB with 108 SMs, driver 535.129.03, PyTorch 2.5.1 with CUDA 12.4, and Triton 3.1.0. Each primitive call contains two packed LogSumExp update kernels. Widths are randomized within 30-repeat blocks after warm-up. The $n = 1024$ sweep covers widths 1–8, 10, 12, and 16; the other sizes cover 1, 2, 3, 4, 6, 8, 12, and 16. The synthetic inputs use feature dimension 500, column tile 64, and cost scale 1. The earlier MetroPT-3 component calibration uses dimension 15, column tile 128, and cost scale 0.0341; the Packer19 component application uses dimension 64. Thus the synthetic primitive time and the Packer19 correction time refer to different cost computations. The application reconstruction below calibrates its own complete kernel configuration.

OTT-JAX verifies at phase boundaries and rebuckets unfinished problems. The tolerance sweep uses one fixed ImageNet-32 input, ten timing repeats per tolerance, and phase sizes selected on disjoint warm-up inputs: 20 updates at 10^{-3} and 10 updates at the other reported tolerances. Timings cover device-resident execution; loading, transfer, compilation, and the common post-run audit occur outside this interval. An earlier six-repeat study gives $2.60\times$ relative to native vectorized execution and $1.55\times$ relative to its matched fixed-phase baseline.

PyKeOps uses a matrix-free batched operator. Its ImageNet-32 experiment compares static batching, logical masking, and two compacted configurations on the same three seeds. Static and masked execution check every update; compacted execution checks every two or four updates. At $n = 16384, W = 4$, the compacted configurations use mean update counts of 14 and 16, compared with 13 for static batching. On the separate A2D2 workload, compacted execution gives $3.17\times$ relative to sequential carry, with an observed clip range of $2.18\text{--}3.94\times$ and maximum marginal residual 9.97×10^{-4} .

The every-update PyKeOps comparison uses three input seeds at each of $n = 2048$ and $16384, W = 4, \varepsilon = 0.1$, and tolerance 10^{-3} . Targets are ImageNet-32 PCA500 features; Gaussian sources and shared proposals follow the existing coupling-preparation task with 90% support overlap. Three cyclic orders per input give 18 paired rounds. Coupling-preparation time sums proposal, repair, and validation, with zero consumer time and the preceding anchor excluded. All 54 timed method runs pass the backend residual check. One small-input order requires an extra strict fallback step in static and masked execution; the reported per-input summaries include that observation. The environment uses one A100, PyTorch 2.5.1/CUDA 12.4, PyKeOps 2.3, and disabled TF32.

The shared-initialization experiment uses a constructed stream with 90% support overlap over real ImageNet features. Its warm replay includes descriptor, transfer, initialization, correction, and verification costs; compilation, cache loading, and process startup occur before measurement. Across 40 controlled windows, compaction gives $1.42\times$ over static execution and removes 30.6% of problem updates.

A.4 Worker scaling and grouping

Independent GPU workers process different windows without a Sinkhorn collective. The one-to-four-worker throughput experiment holds work per worker fixed and achieves $3.97\text{--}3.98\times$ throughput scaling on held-out MetroPT-3 routes. The eight-worker experiment compares fixed-width and compacted execution at that worker count, with pipeline ratios of $3.11\times$ and $2.16\times$ for its two inputs. Table 10 gives the absolute eight-worker times and separate single-worker support-size comparisons. The reported support-size configurations meet the prespecified consumer criteria on both hosts.

Table 10: Additional MetroPT-3 runtime measurements. Eight-worker rows use fixed work per worker and pipeline time; support-size rows are separate single-worker comparisons summarized by equal-host geometric means. Energy ratios use the corresponding eight-worker endpoint.

Configuration	Fixed (s)	Compacted (s)	Time ratio	Energy ratio
8 workers, higher workload	12.5	4.03	3.11	3.44
8 workers, lower workload	8.00	3.71	2.16	2.29
1 worker, $n = 8192$	—	—	2.32	—
1 worker, $n = 32768$	—	—	1.62	—
Update counts: higher workload 4736 / 1276; lower workload 3008 / 1244.				

Grouping uses a fixed multiset of 32 problems. Offline iteration counts q_t determine the layouts, fixing $\sum_t q_t$ while changing $\sum_w W_{q_{\max}, w}$. Runtime ratios describe the resulting effect for this problem set. The layouts provide a controlled analysis of completion variation.

A.5 Initial batch width

The ImageNet-32 width study uses $n = 4096$, $\varepsilon = 0.1$, $\tau = 10^{-3}$, and checks every five updates. Each width in $\{1, 2, 4, 8, 16\}$ contains 40 recorded windows across eight A100 workers on two hosts. The comparison is sequential project-adapter execution with projected carry against shared-initialization active execution. Its runtime ratio measures the combined execution configuration.

Figure 4 separates padding from runtime. For each recorded window, the ratio $W n_{\max} / \sum_t n_t$ compares hypothetical fixed-width work with its executed active work. Its mean rises from 1.00 at $W = 1$ to 1.44 at $W = 16$. Steady-state sequential/active time rises from $0.979\times$ to $1.85\times$ over these widths. The padding ratios use completion counts within each active run; the timing ratios use the two measured execution configurations.

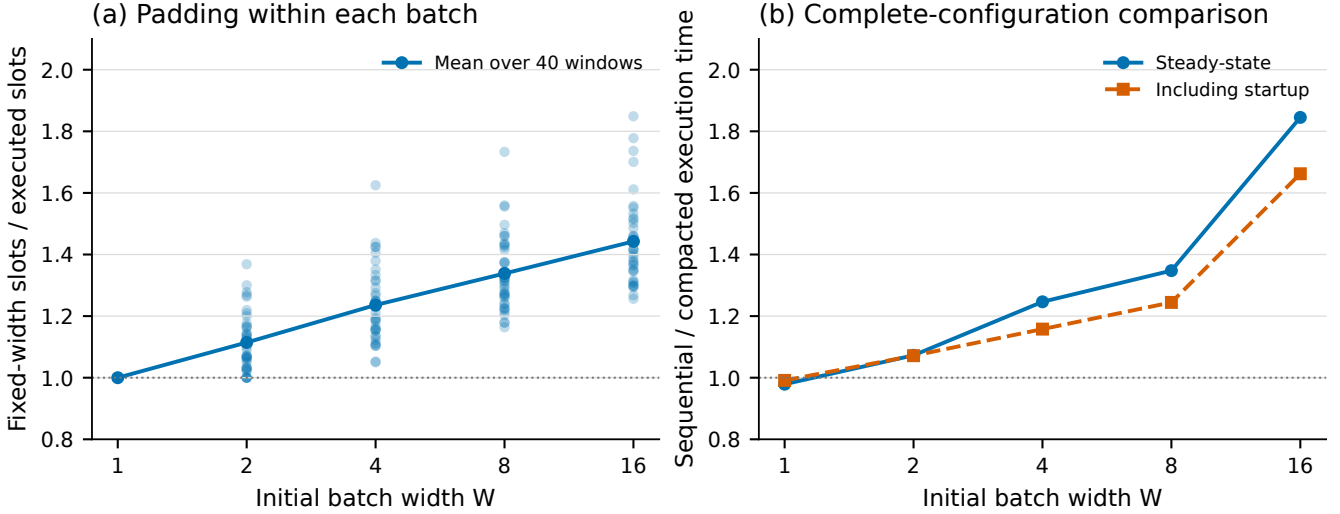


Figure 4: Initial-width sensitivity on ImageNet-32. Left: each point is one window’s $W n_{\max} / \sum_t n_t$; the line joins means over 40 windows per width. Right: ratios of the two eight-worker critical-path times, with and without startup costs. The baseline uses sequential projected carry and the compacted configuration uses shared initialization. Panel (a) describes padding; panel (b) measures their complete execution comparison.

A.6 Finite-precision stress test

An independent test covers 84 near-threshold problems and 16 runtime stress cases over 1/2/4/8 GPUs. Against FP64 replay, raw FP32 and TF32 each disagree on three acceptance decisions. An outward-bound guard followed by FP64 escalation has zero disagreements in these 100 cases. This guarded configuration is evaluated separately from the timed applications.

A.7 Competitive execution and calibration settings

The MetroPT-3 competitive controls use $n = 16384$, feature dimension 15, and $\varepsilon = 0.1$ on one A100-SXM4-80GB. All compared paths use FP32, disabled TF32, exp2, tiles (64, 64, 16), four warps, and two stages. The common cost scale is 0.02822732106894366, fixed from the shared source and first target. Each method checks every four cycles at 9.5×10^{-4} and uses packed updates down to width one. Continuous E_1 timing starts from ready GPU arrays and includes initialization, updates, screening, verification, output capture, control, and final synchronization. FP64 replay and file output follow the timing runs. Separately recorded consumer time is an adjacent measurement.

The four width-four groups contain target indices 0–3, 4–7, 8–11, and 12–15 from the existing failure-window input; the width-eight group contains indices 0–7. Their completion iterations are respectively (356, 312, 320, 272), (140, 32, 32, 32), (28, 32, 100, 172), and (288, 336, 328, 348). They share one source, and distinct target indices can contain overlapping sensor observations. Method order supplies timing repetitions for these fixed batches.

Width-four calibration uses the first group, and width-eight calibration uses the first eight targets. Each method and width has five update/screen measurements after warm-up. Separate instrumented solves measure initialization, verification per candidate, and release-event cost, including empty-event paths. Calibration samples use prefix target IDs at each width. The model sums these costs using each target run’s observed trace and event counts. Error ranges in

table 7 span the three compute-skipping methods and four repeats; they are descriptive ranges. Same-input validation uses separate calibration and timing runs, while target-group transfer also changes the evaluated targets.

The earlier width-16 scheduling experiment uses the same prepared source and targets. Its policies keep, delay, or bucket the physical capacity after verified release, with four-cycle graph blocks. Cost calibration measures update-plus-screen blocks and workspace movement; the solver separately records update, screen, verifier, and movement time. Compilation, graph capture, and calibration precede steady-state solver timing. These measurements support the scheduling comparison reported in section 5.7.

A.8 Reproducibility

The experiment artifacts record input and source identities, software and GPU environments, commands, timings, and output checks. Source code is hosted at <https://github.com/cis-acniticacid/DrainSinkhorn>. Source-linked numerical summaries, figure sources, and reanalysis scripts accompany the manuscript.